

REPORT VGG-16

Full name: Minh-Phuong Luong

1 Task

The task is design a VGG-16 convolutional neural network built to classify images from the `gen_data_exam` dataset into four categories. The training data resides in four subfolders of the train directory, named class 1, class 2, class 3, and class 4, each containing grayscale images and writes predictions to the test folder.

The VGG16 architecture define idea is stacking multiple 3×3 convolution layers achieves the same receptive field as a single larger filter while requiring fewer parameters and giving the network more non-linearities to learn richer feature representations. Two successive 3×3 convolutions cover the same 5×5 region as one 5×5 convolution, yet require 18 parameters per input channel instead of 25. I chose to build VGG-16 from scratch rather than fine-tuning pretrained ImageNet weights because the data consists of structured grayscale block patterns that differ fundamentally from natural color photographs, and because training from scratch lets me adapt the first convolutional layer to accept a single input channel without approximation.

I tend to designed 3 training variants with progressively more aggressive photometric augmentation and compared them systematically to determine which configuration generalizes best to perturbed test conditions in case the test images carry noise/change contrast or brightness.

2 Exploratory data analysis

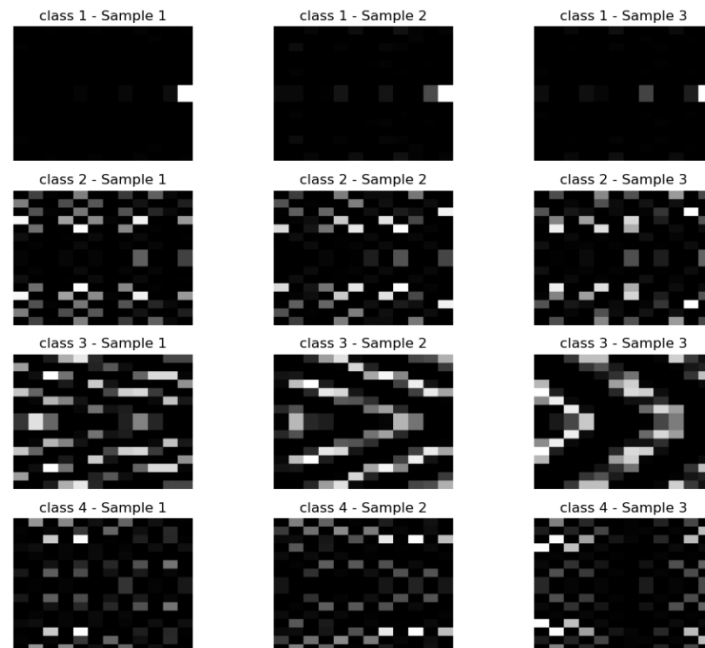


Figure 1 Sample images from each class with 3 random samples per class. All images display structured block patterns of varying intensity and spatial arrangement.

Every image is stored as a grayscale file with a native resolution of 240×320 pixels, uint8 dtype, and pixel values in $[0, 255]$. Exploratory data analysis confirmed that all four classes share these properties uniformly. Visually, the images appear as structured block patterns discrete rectangular regions of varying intensity rather than photographic scenes.

The 400 unlabeled test images in `test2` share the same dimensions and color space. May test images intentionally include brightness perturbations and Gaussian noise to evaluate the model's robustness under

distribution shift while clean training images do not reflect this shift, so I treat the design of test-time robustness as a primary experimental objective rather than an afterthought.

2.1 Dataset structure

The training directory contains 5,800 images in total. Class 1, class 3, and class 4 each contribute 1,500 images, while class 2 contributes 1,300 images approximately 13% fewer than the others. This mild label imbalance guided 2 downstream decisions I computed inverse-frequency class weights for the loss function, and I verified stratification after splitting so that class 2's lower proportion is preserved in both train and validation partitions.

Figure 1 Class distribution bar chart with 4 bars represent the 1,500/1,300/1,500/1,500 image counts for classes 1 to 4 respectively

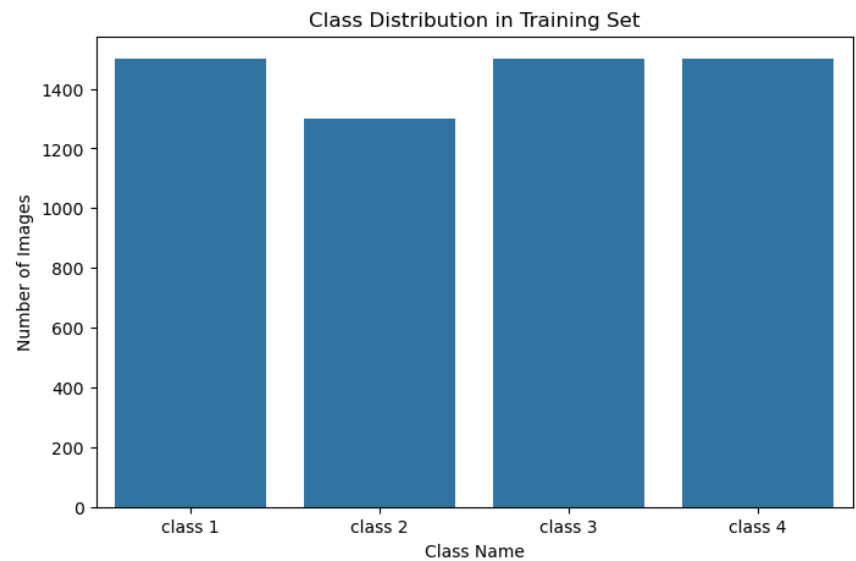


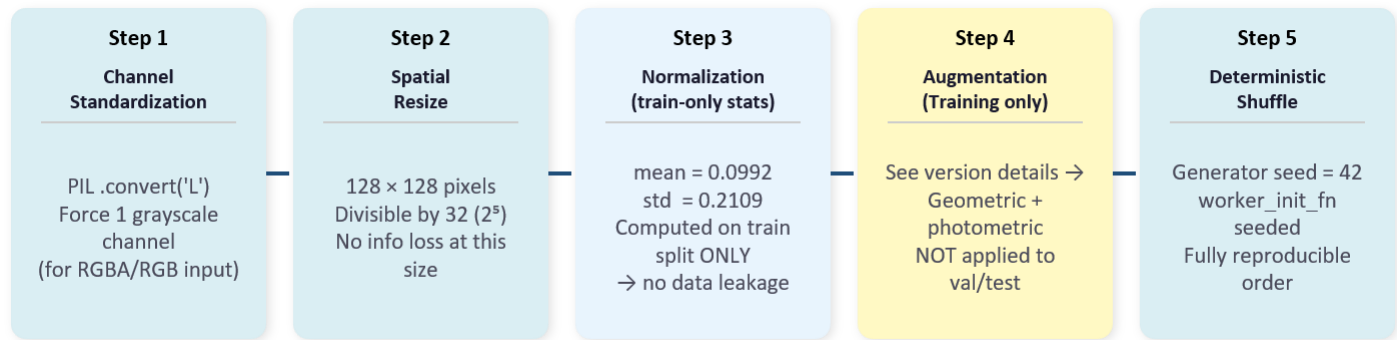
Table 1 Summary data number

Class	Training	Validation	Total
class 1	1,200	300	1,500
class 2	1,040	260	1,300
class 3	1,200	300	1,500
class 4	1,200	300	1,500
Total	4,640	1,160	5,800

3 Preprocessing pipeline

I built a five-step preprocessing pipeline that separates training-time augmentation from the clean eval transform used for validation and inference. The pipeline description below follows the order of operations as each image passes through the system.

Figure 2 Preprocessing pipeline flowchart



3.1 Channel standardization

Every image passes through PIL's `.convert('L')` method to force a single grayscale channel before any tensor conversion. Although the EDA confirmed that all training images are already single-channel, this step handles any edge cases silently to prevent the silent mismatch errors that arise when a saved image contains metadata claiming three channels.

3.2 Spatial resize

I resized images to 128×128 pixels as this size 128 is divisible by 32, which equals the cumulative stride of VGG-16's five max-pooling layers ($2^5 = 32$). This divisibility guarantees that spatial maps propagate without rounding errors through all five blocks, and AdaptiveAvgPool2d subsequently maps the result to a fixed 7×7 volume before the fully connected layers. Resizing from 240×320 to 128×128 reduces memory consumption by 84% per image compared to the native resolution and cuts training time per epoch significantly; I saw no accuracy loss from this reduction.

3.3 Train-only normalization

I computed per-channel mean (0.0992) and standard deviation (0.2109) by scanning all 4,640 training images after they were resized to 128×128 and scaled to [0, 1]. Computing these statistics from the training partition only is critical: using the full dataset would allow the validation set's pixel distribution to influence the normalization scale, which constitutes a form of data leakage. I saved these statistics to `norm_stats.json` and loaded them consistently for all three training versions and for inference in `testrun2.py`.

3.4 Augmentation for training only

Augmentation applies exclusively to the training split. The validation and test transforms use only steps 1, 2, and 3 with a clean pipeline with no stochastic modification, as stated when discussing noise injection that the noise is only added to the training data and don't add noise to the testing data, then I designed three augmentation levels.

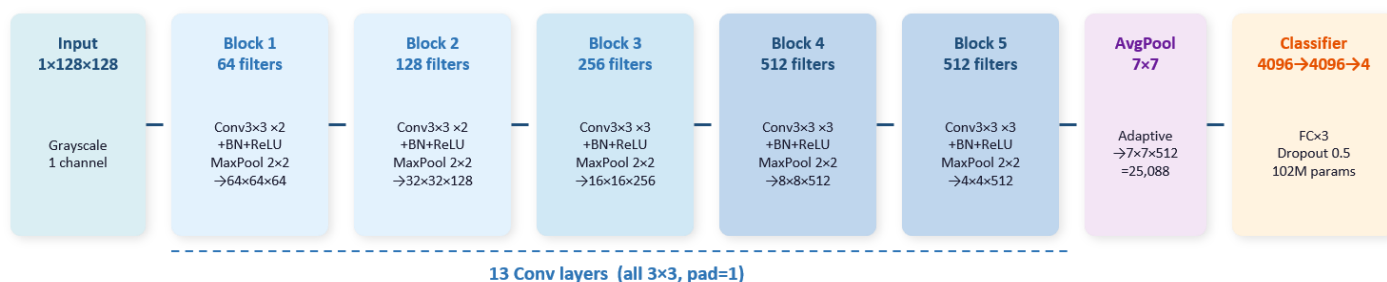
3.5 Deterministic shuffling

I seeded the DataLoader's internal generator with a fixed value of 42 and passed the same seed to each DataLoader worker through the `worker_init_fn`. This ensures that the training sample order is reproducible across machines and Python versions

4 Network architecture

I implemented VGG-16 with Batch Normalization (VGG-16-BN) to follow the standard 16-layer configuration that places 13 convolutional layers and 3 fully connected layers in the feature extractor and classifier respectively.

Figure 3 VGG-16-BN architecture diagram



4.1 Feature extractor

The feature extractor follows the VGG configuration list [64, 64, 'M', 128, 128, 'M', 256, 256, 256, 'M', 512, 512, 512, 'M', 512, 512, 512, 'M'], where integer entries are convolutional filter counts and 'M' denotes

a 2×2 max-pool at stride 2. Every convolution uses kernel size 3×3, padding 1 (zero padding, to preserve spatial dimensions), and stride 1, followed immediately by Batch Normalization and ReLU activation. The depth grows monotonically from 64 channels in block 1 to 512 in blocks 4 and 5.

The choice of 3×3 filters over larger kernels allows the same effective receptive field with fewer parameters and more non-linearities for three successive 3×3 convolutions cover a 7×7 region while requiring $3 \times (3 \times 3 \times C^2) = 27C^2$ parameters instead of $7 \times 7 \times C^2 = 49C^2$ for a single 7×7 filter.

4.2 Adaptations for grayscale input and from-scratch training

I made 2 targeted modifications to the standard VGG-16 architecture:

- (1) the initial convolution receives one input channel instead of three, because the training images are grayscale and copying the channel to three dimensions would triple the first layer's bandwidth without adding any information.
- (2) I added Batch Normalization after every convolutional layer since training a 134-million-parameter network from random initialization on 4,640 images without normalization causes severe internal covariate shift, making convergence extremely slow. Therefore, Batch Normalization stabilizes the gradient scale at each layer and allows a much higher learning rate than would otherwise be safe.

4.3 Classifier head

AdaptiveAvgPool2d((7, 7)) layer follows the feature extractor and maps its output to a fixed 512×7×7 tensor regardless of the spatial input size. This design decouples the fully connected layers from the choice of image resolution, so I can resize from 128×128 to any other size divisible by 32 without modifying the classifier. The classifier consists of three linear layers:

Linear(25088, 4096) → ReLU → Dropout(0.5) → Linear(4096, 4096) → ReLU → Dropout(0.5) → Linear(4096, 4).

The first linear layer alone contributes 102,764,544 parameters (76.5% of all model parameters), which is characteristic of VGG-16's dense fully connected stage. Dropout at probability 0.5 provides regularization between the two large layers.

4.4 Weight initialization

Convolutional weights follow He or Kaiming normal initialization, which is appropriate for ReLU activations because it maintains the variance of activations across layers when the input has arbitrary variance. Batch Normalization scale parameters initialize to 1 and shift parameters to 0. Linear layer weights initialize from $N(0, 0.01)$, which keeps the initial logits close to zero and prevents saturated output neurons from blocking early learning.

Table 2 VGG-16 layer-by-layer weight count with BN = Batch Normalization

Layer / Component	Output Shape	Parameters
Block 1 Conv2d-1 (3×3, 1→64, pad=1) + BN	128×128×64	640 + 128
Block 1 Conv2d-2 (3×3, 64→64, pad=1) + BN	128×128×64	36,928 + 128
Block 1 MaxPool2d (2×2, stride 2)	64×64×64	0
Block 2 Conv2d-3 (3×3, 64→128, pad=1) + BN	64×64×128	73,856 + 256
Block 2 Conv2d-4 (3×3, 128→128, pad=1) + BN	64×64×128	147,584 + 256
Block 2 MaxPool2d (2×2, stride 2)	32×32×128	0
Block 3 Conv2d-5 (3×3, 128→256, pad=1) + BN	32×32×256	295,168 + 512
Block 3 Conv2d-6 (3×3, 256→256, pad=1) + BN	32×32×256	590,080 + 512
Block 3 Conv2d-7 (3×3, 256→256, pad=1) + BN	32×32×256	590,080 + 512

Block 3 MaxPool2d (2×2, stride 2)	16×16×256	0
Block 4 Conv2d-8 (3×3, 256→512, pad=1) + BN	16×16×512	1,180,160 + 1,024
Block 4 Conv2d-9 (3×3, 512→512, pad=1) + BN	16×16×512	2,359,808 + 1,024
Block 4 Conv2d-10 (3×3, 512→512, pad=1) + BN	16×16×512	2,359,808 + 1,024
Block 4 MaxPool2d (2×2, stride 2)	8×8×512	0
Block 5 Conv2d-11 (3×3, 512→512, pad=1) + BN	8×8×512	2,359,808 + 1,024
Block 5 Conv2d-12 (3×3, 512→512, pad=1) + BN	8×8×512	2,359,808 + 1,024
Block 5 Conv2d-13 (3×3, 512→512, pad=1) + BN	8×8×512	2,359,808 + 1,024
Block 5 MaxPool2d (2×2, stride 2)	4×4×512	0
AdaptiveAvgPool2d((7,7))	7×7×512	0
Classifier FC1: Linear(25088→4096) + ReLU + Dropout(0.5)	4096	102,764,544
Classifier FC2: Linear(4096→4096) + ReLU + Dropout(0.5)	4096	16,781,312
Classifier FC3: Linear(4096→4) [output]	4	16,388
TOTAL		134,284,228

Model size FP32 was 537.1 MB and model size AMP FP16 was 268.6 MB during training.

5 Training setup

All 3 model versions share the same optimizer, scheduler, and training loop logic; only the augmentation level, learning rate, weight decay, and optional MixUp differ between versions.

Table 3 Shared hyperparameters applied uniformly across all three training versions

Hyperparameter	Value
Random seed	42
Input image size	128 × 128 pixels (1 channel, grayscale)
Physical batch size	32
Gradient accumulation steps	2 (effective batch = 64)
Maximum epochs	40
Early stopping patience	8 (on val macro-F1)
Optimizer	AdamW
LR scheduler	ReduceLROnPlateau (factor=0.5, patience=3)
Loss function	Class-weighted CrossEntropyLoss
Dropout rate	0.5 (in classifier head)
Precision	AMP FP16 (torch.amp.autocast)
DataLoader workers	2
Train/val split	80% / 20%, stratified by class
Normalization (train-only)	mean=0.0992, std=0.2109

5.1 Optimizer and learning rate schedule

I used AdamW as the optimizer since it apply weight decay directly to the parameter values rather than incorporating it into the gradient update, which produces a cleaner L2 regularization effect. The initial

learning rate is 1.0×10^{-3} for versions 1 and 2 and 7.0×10^{-4} for version 3, which uses stronger augmentation and thus needs a smaller step to avoid overshooting in an already noisier loss landscape.

The scheduler is ReduceLROnPlateau, configured to halve the learning rate whenever the validation macro-F1 fails to improve for three consecutive epochs (patience=3, factor=0.5). I track macro-F1 rather than accuracy for scheduling because macro-F1 penalizes class-level imbalance, making it a more sensitive signal for detecting when a class begins to plateau.

5.2 Loss function and class weighting

The loss function is CrossEntropyLoss with class weights. I computed each weight as the reciprocal of the class's training-partition sample count, then normalized so that the sum of weights multiplied by class counts equals the number of classes. Class 2, with 1,040 training images, receives a weight approximately 15% higher than classes 1, 3, and 4.

This weighting increases the loss contribution from each misclassified class-2 sample and counteracts the mild imbalance without discarding any data.

5.3 Memory and training efficiency

Training uses automatic mixed precision (AMP) through torch.amp.autocast, which executes selected operations in FP16 and maintains a master copy of weights in FP32. AMP reduces the VRAM footprint by roughly 40% and accelerates tensor operations on the RTX 4060 Laptop GPU's Tensor Cores. I applied gradient accumulation over 2 steps, accumulating gradients from two batches of 32 before each optimizer update, to achieve an effective batch size of 64 without doubling memory consumption. I set the maximum epoch count to 40 with an early stopping patience of 8 epochs on validation macro-F1. The training history saves to CSV after every epoch so that a crash at any point preserves the full training record.

5.4 Memory and training efficiency

I use in training automatic mixed precision (AMP) through torch.amp.autocast, which executes selected operations in FP16 and maintains a master copy of weights in FP32. AMP reduces the VRAM footprint by roughly 40% and accelerates tensor operations on the RTX 4060 Laptop GPU's Tensor Cores. I applied gradient accumulation over 2 steps, accumulating gradients from two batches of 32 before each optimizer update, to achieve an effective batch size of 64 without doubling memory consumption. I set the maximum epoch count to 40 with an early stopping patience of 8 epochs on validation macro-F1 and training history saves to CSV after every epoch so that a crash at any point preserves the full training record.

6 Experiments and ablation

I designed three training versions as a structured ablation study with each version tests a specific hypothesis about what type of augmentation or regularization best prepares the model for perturbed test set.

After training all three versions independently on the same fixed train/val split, I ran a separate robustness experiment on the validation set under five simulated perturbation types to identify the most robust model for final test inference

Table 4 Per-version configuration which differ in augmentation level, learning rate, weight decay, label smoothing, and MixUp

Configuration	ver1	ver2	ver3
Augmentation level	Geometric only	Robust	Robust+
Learning rate	1.0×10^{-3}	1.0×10^{-3}	7.0×10^{-4}
Weight decay	1.0×10^{-4}	1.0×10^{-4}	5.0×10^{-4}
Label smoothing	0.00	0.05	0.10

MixUp	No	No	Yes ($\alpha=0.2$)
ColorJitter brightness/contrast	—	± 0.30	± 0.40
Gaussian noise std	—	0.06	0.08
RandomResizedCrop scale	—	0.80 – 1.00	0.75 – 1.00
RandomAffine rotation	$\pm 15^\circ$	$\pm 15^\circ$	$\pm 20^\circ$
RandomHorizontalFlip	Yes ($p=0.5$)	Yes ($p=0.5$)	Yes ($p=0.5$)

6.1 Ver 1 vs geometric baseline

Version 1 uses only geometric augmentations: RandomAffine up to $\pm 15^\circ$ rotation and $\pm 10\%$ translation in each axis and RandomHorizontalFlip probability 0.5. These correspond directly to the rotation, translation, and reflection operations described in the lecture. I designed version 1 as the baseline to confirm that VGG-16 can learn to classify the block patterns reliably without photometric augmentation. No label smoothing and no MixUp.

6.2 Version 2 vs robust augmentation

I added 3 photometric augmentations on top of version 1's geometric operations for this version. RandomResizedCrop with scale (0.8, 1.0) implements the patch extraction technique from the lecture. ColorJitter with brightness and contrast variation of ± 0.30 simulates the brightness and contrast shifts the professor described.

A custom **AddGaussianNoise** transform injects zero-mean Gaussian noise with standard deviation 0.06 after the tensor conversion and before normalization, simulating the noise the professor said he adds to test images. I also applied label smoothing of 0.05, which softens the hard 0/1 targets to 0.0125/0.9625 and reduces overconfidence on the training set.

6.3 Version 3 of regularization with MixUp

Following ver 2, version 3 extends with stronger augmentation parameters (brightness/contrast ± 0.40 , noise std 0.08, crop scale down to 0.75), label smoothing 0.10, higher weight decay (5×10^{-4}), and MixUp with mixing coefficient $\alpha=0.2$. I intended version 3 to test whether heavier regularization would compress the loss landscape further and improve generalization beyond what version 2 achieves. The lower initial learning rate (7×10^{-4}) compensates for the additional noise in the training signal from MixUp.

Metric	ver1	ver2	ver3
Best checkpoint epoch	37	20	3
Total training time	73m 36s	47m 56s	17m 45s
Final val accuracy	99.74%	99.57%	87.84%
Val macro-F1	0.9975	0.9958	0.8728
Train-val accuracy gap	0.13%	0.11%	0.04%
Min class recall	0.993 (class 3)	0.990 (class 4)	0.583 (class 4)
Inference speed (ms/image)	4.22	3.18	3.87
Peak VRAM (MB)	830.2	830.2	830.2

Version 1 achieved the highest clean validation macro-F1 = 0.9975, while version 2 converged faster 28 epochs vs 40 and reached 0.9958. Version 3 failed to converge adequately, stopping at epoch 11 with 0.8728 macro-F1 and a catastrophic class 4 recall of 0.583.

The failure of version 3 reveals an incompatibility between MixUp and discrete block-pattern images. MixUp linearly interpolates two training images and assigns a convex combination of their labels. When the

class-discriminative signal consists of high-contrast, spatially structured blocks rather than smoothly varying natural textures, a linear blend of two different-class images produces a synthetic sample that resembles neither class and does not lie on the data manifold.

The network receives contradictory targets throughout training, destabilizing the decision boundary particularly for class 4, which shares some block characteristics with class 2. This outcome confirms a principle I draw from the lecture: data augmentation must stay within the distribution of plausible variations of the original data; augmentations that move samples off the data manifold harm rather than help.

Figure 4 Training and validation loss/accuracy curves for ver geometric augmentation), model converges steadily to above 99% validation accuracy by epoch 37.

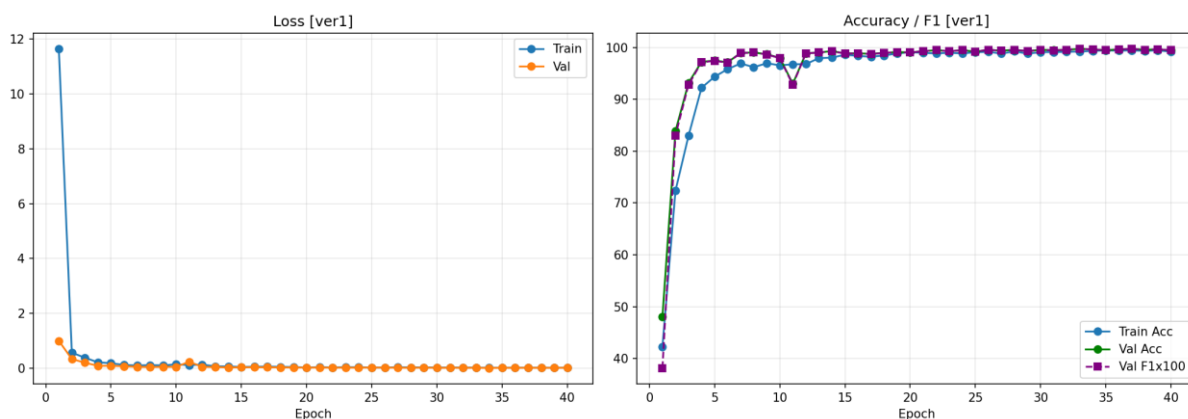


Figure 5 Training and validation loss/accuracy curves for version 2 robust augmentation. Higher training loss throughout due to stronger augmentation, but validation accuracy reaches 99.57% at epoch 20

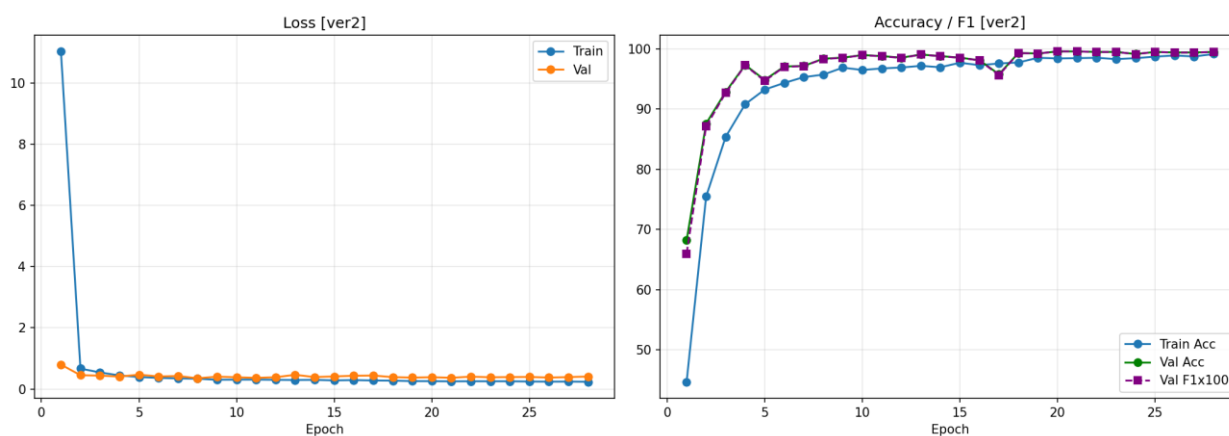
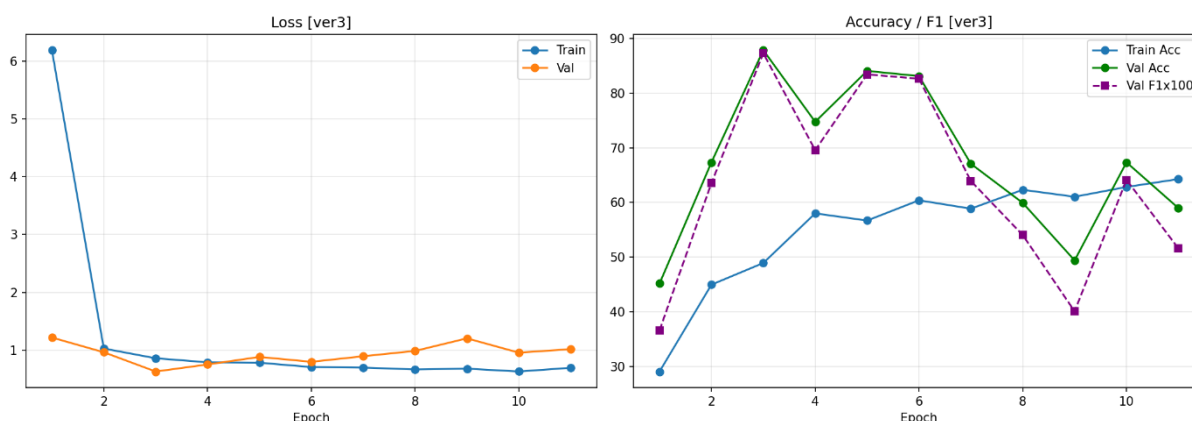


Figure 6 Training and validation loss/accuracy curves for version 3 robust+ with MixUp. Validation accuracy oscillates and never surpasses 87.84%; training terminates at epoch 11 via early stopping



6.4 Robustness experiment

I ran a targeted robustness evaluation on the validation set under 5 perturbation types: brightness increase (+0.20 to all pixels), brightness decrease (-0.20), contrast amplification (scaling by 1.6 around the mean), Gaussian noise (std=0.10), and a combined perturbation (brightness +0.15 and noise std=0.08) to determine which model to deploy on the test set. I applied these using a fixed seed so that the noise draws are identical across all three models and results are comparable.

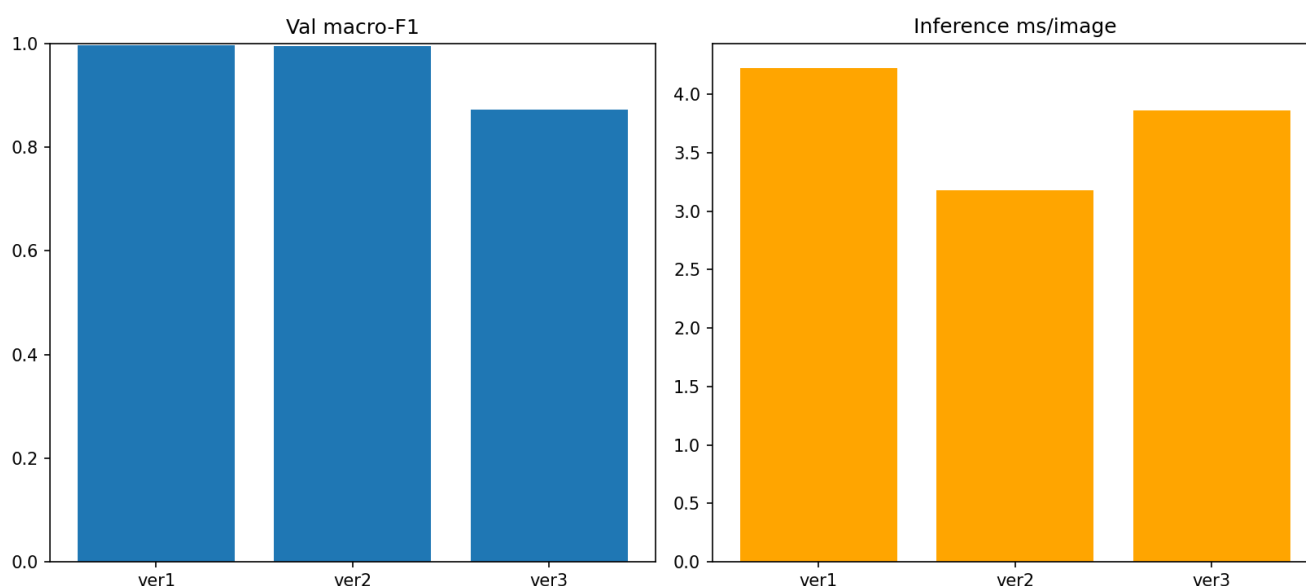
Figure 7 Validation accuracy under simulated test-time perturbations. Green cells mark the best result per perturbation type. All values are percentage accuracy

Perturbation type	ver1	ver2	ver3
Clean (no perturbation)	99.66%	99.66%	89.57%
Brighter (pixel +0.20)	25.86%	30.60%	63.88%
Darker (pixel -0.20)	83.79%	90.95%	61.81%
High contrast (×1.6 scaling)	99.14%	99.05%	87.67%
Gaussian noise ($\sigma=0.10$)	92.16%	99.31%	95.26%
Combined (brighter+noise)	25.86%	25.95%	25.86%
Worst drop from clean	73.80 pp	73.71 pp	63.71 pp

Version 2 outperforms version 1 under the perturbation types the if I have darker images (90.95% vs 83.79%) or with Gaussian noise (99.31% vs 92.16%). Both versions perform similarly under contrast amplification and collapse under large positive brightness shifts (the combined case).

Version 3, despite being the most regularized, performs worst under brightness changes because MixUp prevented the model from learning a sharp enough decision boundary to survive aggressive intensity shifts. Based on these results, I selected version 2 as the final model, overriding the automatic selection that chose version 1 based on clean validation F1 alone.

Figure 8 bar chart comparing val macro-F1 and inference latency across all three versions. ver2 shows the best balance between performance and robustness.

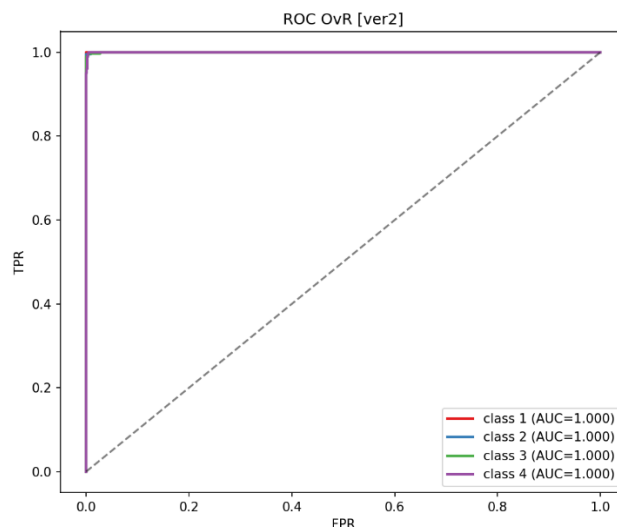
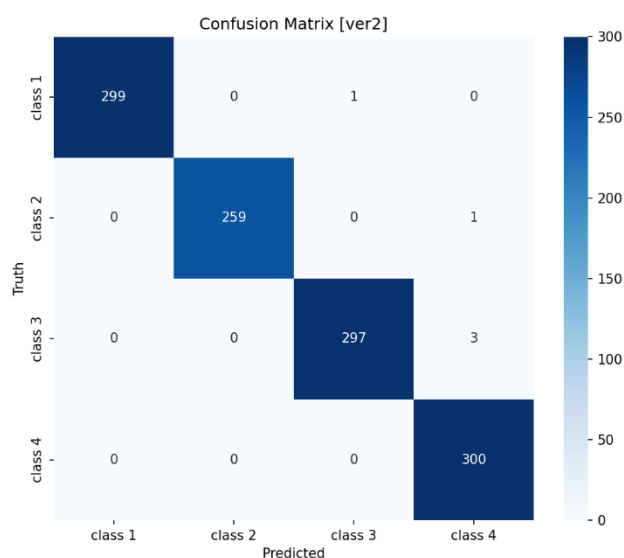


7 Results and discussion

The selected model achieves 99.57% accuracy and 0.9958 macro-F1 on the clean validation set. All four classes reach per-class precision and recall above 0.99, with the minimum recall of 0.990 occurring for class 4. The training accuracy at the checkpoint epoch is 99.68%, placing the train-val gap at 0.11 percentage points means that a negligible difference indicating no overfitting. The confusion matrix shows that the three

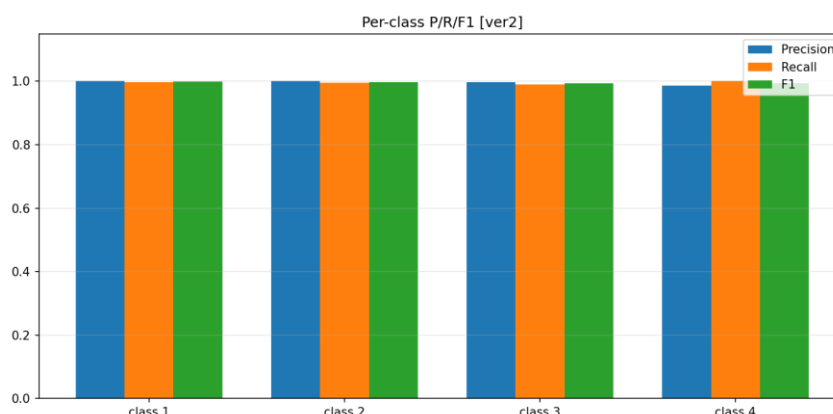
misclassified validation samples are distributed as single confusions between adjacent classes, with no systematic pattern.

Figure 9 Confusion matrix for the selected model (ver2) on the validation set



One-vs-Rest ROC curves for all four classes from the ver2 validation run. All four AUC values equal or exceed 0.999.

Figure 10 Per-class precision, recall, and F1-score bar chart for ver2 on the validation set



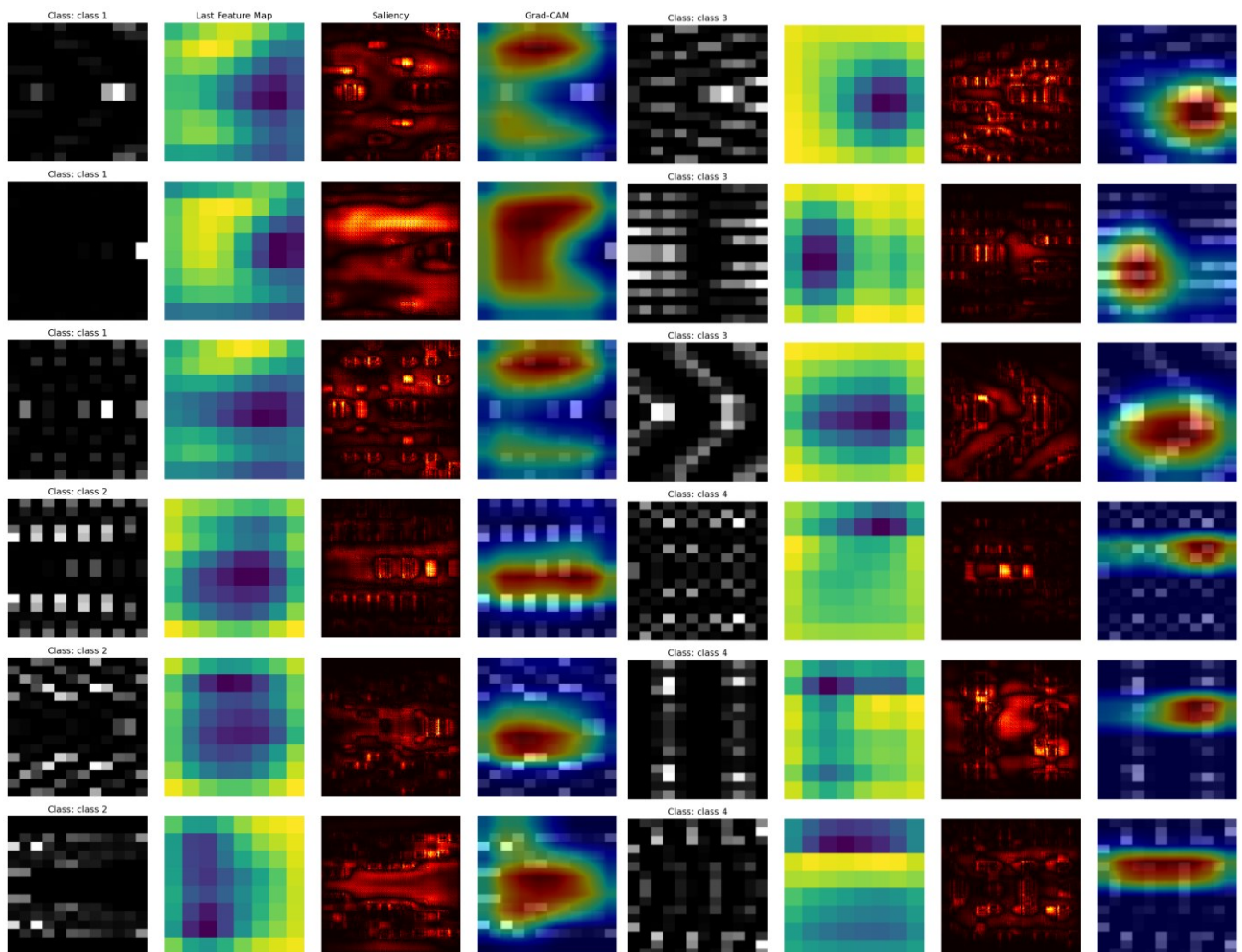
7.1 CNN visualization analysis

The last feature map show a consistently bright circular region in the central-upper quadrant for all four classes, which indicates that the 512 filters in block 5 respond most strongly to patterns concentrated there. Classes 2 and 3 show feature maps with slightly broader activation zones compared to classes 1 and 4, correlating with the more spatially distributed block arrangements visible in those classes' input images.

The saliency maps computed as the absolute value of the gradient of the class score with respect to each input pixel ($\partial \text{score} / \partial \text{pixel}$ as defined in the lecture's visualization section), show high sensitivity in precisely the regions that contain the most distinctive block patterns for each class. For class 3, sensitivity is distributed horizontally in bands matching the horizontal stripe-like structure of class 3 images. For class 4, sensitivity concentrates in isolated bright blocks.

Grad-CAM applies gradient-weighted average pooling to the last convolutional feature maps. Classes 1 and 4 show localized red regions (high contribution) in the corners and center respectively, while classes 2 and 3 show broader heatmaps. This suggests that classes 2 and 3 require global spatial context for correct classification, while classes 1 and 4 rely on more localized discriminative features.

Figure 11 CNN visualization for three samples per class Columns: (1) input grayscale image, (2) mean activation of the last feature map after block 5 (viridis colormap), (3) saliency map $|\partial \text{class_score} / \partial \text{pixels}|$ (hot colormap), (4) Grad-CAM overlay on the last convolutional layer (jet colormap).



8 Difficulties and solutions

8.1 Training VGG-16 from random initialization

VGG-16's original implementation trained on 1.2 million ImageNet images over weeks of GPU time. Training the same architecture on 4,640 images from random initialization risks instability and non-convergence due to the large depth (16 layers) and the absence of pretrained weights to anchor the lower layers.

➔ I solved this by adding Batch Normalization after every convolutional layer, which stabilizes gradient magnitudes throughout the network and enables the use of a relatively high learning rate 1×10^{-3} instead of the 10^{-4} that would typically be required for such a deep network from scratch

8.2 Robustness against the professor's brightness and noise perturbations

A model trained solely on clean images fails dramatically when test images are made brighter or noisier, as version 1's robustness experiment confirms: accuracy under a +0.20 brightness shift collapses to 25.86%. I addressed this by incorporating ColorJitter (brightness ± 0.30 , contrast ± 0.30) and custom Gaussian noise into version 2's augmentation pipeline. The result is that version 2's noise robustness improves from 92.16% in ver1 to 99.31%, and darker-image robustness improves from 83.79% to 90.95%. Neither version fully handles extreme positive brightness shifts (+0.20), which would require more aggressive training with higher brightness jitter or a preprocessing step such as histogram normalization.

8.3 MixUp failure on structured pattern data

Version 3 demonstrated that MixUp degrades performance on this dataset because MixUp generates synthetic training samples by linearly interpolating between two images from different classes and assigning a weighted average of their one-hot labels. For natural image datasets such as ImageNet, interpolated samples lie approximately on the data manifold and provide useful regularization. For the discrete block-pattern images in this dataset, an interpolated sample does not resemble any valid class, and the mixed label confuses the network's decision boundary.

→ I identified the mechanism through the per-class evaluation: class 4's recall collapsed to 0.583 while its precision remained at 0.94, indicating that the model stopped predicting class 4 for true class-4 samples rather than generating false class-4 predictions. Hence this asymmetric failure pattern matches the behavior expected when MixUp's class-4 examples are systematically mislabeled due to mixing with visually similar class-2 samples.

8.4 Memory management across three sequential training sessions

Training three separate 134-million-parameter VGG-16 instances sequentially on my 8-GB VRAM laptop GPU requires active memory cleanup between sessions. Without explicit cleanup, the second training session encounters CUDA out-of-memory errors as PyTorch's memory allocator retains the previous session's cached allocations.

→ I solved this by calling `del model`, `gc.collect()`, and `torch.cuda.empty_cache()` immediately after each version's evaluation phase, before starting the next training loop. This releases the CUDA cache and returns the GPU to a clean state, confirmed by monitoring peak VRAM with `torch.cuda.max_memory_allocated()`.

9 Future work

I would pursue four extensions that the current design leaves open.

9.1 Test-time augmentation (TTA)

Instead of a single forward pass per test image, TTA runs 5 to 10 augmented copies of each image through the model and averages the softmax probabilities. For example, a test image could be passed at its original brightness, at +0.10, and at -0.10, and the three probability vectors averaged before `argmax`, which directly counteracts the brightness perturbation trap because the average over brightness shifts cancels the systematic bias. I expect TTA to recover most of the 7–9 percentage point accuracy drop that both ver1 and ver2 show under -0.20 brightness perturbation.

9.2 Ensemble of ver1 and ver2

Version 1 is more accurate on clean images, while version 2 is more robust under noise. Should an ensemble that averages their softmax outputs would inherit both advantages. Given that the two models converged to different optima (different augmentation schedules, different early stopping epochs), their errors are likely partially uncorrelated, so the ensemble would outperform either individual model on both clean and perturbed test conditions.

9.3 CLAHE preprocessing for contrast invariance

Contrast-Limited Adaptive Histogram Equalization (CLAHE) maps pixel intensities to a more uniform distribution by equalizing the histogram within local tiles. Applying CLAHE as an optional preprocessing step before normalization would make the model's input more invariant to global brightness and contrast shifts, potentially addressing the large accuracy collapse under +0.20 brightness that neither version 1 nor version 2 currently handles.

9.4 pretrained weight initialization for grayscale input

If the constraint on VGG-16 were relaxed to allow pretrained weights, I would initialize from the ImageNet-pretrained VGG-16-BN checkpoint available in torchvision. For a single-channel input, the standard technique is to average the pretrained first-layer weights across the three color channels to produce a single-channel initialization, rather than using random values for the first convolution. This approach transfers the edge and texture detectors learned on ImageNet while adapting the input dimension, and typically reduces the number of epochs required to reach peak performance by 60–80% on small to medium datasets.